GitHub Copilot app for developers

Scope it, isolate it, review it, ship it

Session M2-05 | Module 2 | Intermediate

Agenda

Segment	Time
What the app is	8 min
Sessions, chats, and workspaces	12 min
Modes, permissions, sandboxes	15 min
Demo: bug to PR	12 min
Commands, handoff, pros and cons	8 min
Lab handoff	5 min

What the app is

A desktop app for **directing** coding agents.

- Built on the Copilot CLI. Skills and instructions carry over.
- Many sessions at once, each on its own branch.
- Diff view, issues, pull requests, and checks in one place.
- Runs on **This computer** or in a **Cloud** sandbox.

You describe the task. You review the result.

App, IDE, or CLI?

	App	IDE agent mode	CLI
Works in	Own worktree	Your open checkout	Current folder
Parallel	Built in	Awkward	You manage folders
Strong at	Review, PRs, supervising	Editing with you	Scripts, quick tasks
Weak at	One-line fixes	Collides with your edits	No review UI

Pick by task size and how much you need to **review**.

Chat or session?

	Chat	Session
Has a branch	No	Yes
Edits files	No	Yes
Use it to	Think, compare, explain	Change and ship

Switching tasks? **Start a new session.** Old context hurts new work.

Workspace: what a worktree is

A **git worktree** is a second folder for the same repo. It has its own branch and files. It shares history.

```
git worktree list
```

Workspace option	Meaning
New worktree	Own folder and branch. Safe in parallel.
Current checkout	Your folder. One session only.

Two agents in one folder overwrite each other. Two worktrees don't.

Worktree traps

- **Tracked files only.** No `.env`, no `node_modules`. Tell the agent how to set up.
- **Disk.** Archive finished sessions.
- **Merge conflicts still happen.** Isolation stops overwrites, not overlap.
- **One branch, one worktree.** Git won't check out a branch twice.

Three settings, not one

Setting	Controls	Options
Mode	How the agent works with you	Interactive, Plan, Autopilot
Tool permissions	When it asks before a tool	Ask every time, Assisted, Approve all
Sandbox	What tools can reach	Local sandbox, cloud sandbox

Autopilot with *Ask every time* still stops for each tool.

The three modes

Mode	Does	Choose when
Interactive	Suggests, waits for you	Task is unclear
Plan	Plans first, you approve	Many files, costly mistakes
Autopilot	Codes, tests, retries alone	Task is clear and tests exist

Switch any time: `/interactive`, `/plan`, `/autopilot`.

How each mode fails

Mode	Cost	Failure
Interactive	You are the bottleneck	Rubber-stamp approvals
Plan	Extra review step	Plausible plan, wrong problem
Autopilot	You see results, not steps	Runs far on a wrong assumption

Autopilot is for clear problems, not hard ones.

Common pairing: **Plan**, then **Autopilot** with **Assisted** permissions.

Demo: bug to PR

1. **Chat:** paste the issue. What's ambiguous?
2. **Session:** New worktree, Plan mode, `#` the issue.
3. **Read the plan.** Ask for a revision. Approve.
4. `git worktree list` while it runs.
5. **Review:** diff, run the tests yourself, `/review`.
6. **`/pr-open`**. Check the PR and its check on GitHub.

Commands that exist

Command	Use
`/plan`, `/interactive`, `/autopilot`	Switch mode
`/review`, `/rubber-duck`, `/spar`	Critique
`/security-review`	Security pass on the diff (preview)
`/fork`, `/merge-to-parent`, `/spawn`	Branch a session
`/pr-open`, `/pr-fix-checks`, `/pr-resolve-comments`, `/pr-merge`	PR steps
`/compact`, `/clear`, `/context`	Manage context

Names change. **Check `/help` in your version.**

Handing off work

Need	Use
Try an alternative	`/fork`, then `/merge-to-parent`
Side task	`/spawn`
Shorten history	`/compact`
New task	New session plus a three-line brief
Follow from elsewhere	Cloud session or `/remote` (policy permitting)

Brief: **goal, what's done, what's next.** Name the branch and the failing test.

Pros and cons

Pros	Cons
Parallel work without collisions	Heavy for small edits
Diff, checks, PR in one place	More sessions means more review
Start from an issue	Commands and previews change
Autonomy per session	Worktrees need setup and disk

Common mistakes

- Trusting "done". **Run the tests yourself.**
- Autopilot on a vague prompt. **Plan first.**
- **Approve all** on a real repo.
- Two sessions in one checkout.
- Agent edits tests until they pass. **Say no in the prompt. Read the diff.**

Lab: fix a bug and open a PR

You will:

1. Create a **throwaway** GitHub repo and issue.
2. Scope the bug in a chat.
3. Fix it in a **Plan** session in a new worktree.
4. Run a second session in parallel.
5. Review: diff, tests, `/review`.
6. Open the PR, read the check.
7. Redo a small task in the CLI or IDE. Compare.

Starter: `lab/starter`. Four failing tests.